Laporan TA Modul06 LaporBanjir



Oleh:

Nama	:	Zulka Ridho Mahmudi
NIM	:	707012500018
Kelas	:	D4 SIKC 49-04
Mata Kuliah	:	Pemrograman Aplikasi Berbasis Web
Dosen Pengampu	:	Diash Firdaus

PROGRAM STUDI S.Tr SISTEM INFORMASI KOTA CERDAS
FAKULTAS ILMU TERAPAN
TELKOM UNIVERSITY
BANDUNG

2026

A. Tujuan Praktikum

- Memahami konsep authentication dan authorization pada Laravel.
- Memahami peran session, hashing password, dan middleware.
- Membuat fitur register, login, dan logout secara manual tanpa Breeze.
- Menerapkan validasi input dan penyimpanan akun ke database.
- Melindungi halaman petugas dengan middleware `auth`.
- Membuat akun petugas dummy menggunakan seeder dan Faker.
- Mengintegrasikan fitur LaporBanjir dari Modul 4 ke dalam project Modul 6.
- Membedakan alur warga yang melapor tanpa akun dan petugas yang mengelola laporan.
- Menguji fitur authentication dan pelaporan melalui browser serta database.

B. Alat dan Bahan

B.1 Perangkat Keras

- Laptop dengan prosesor minimal 1 GHz.
- RAM minimal 4 GB, disarankan 8 GB.
- Ruang penyimpanan kosong minimal 2 GB.
- Koneksi internet untuk Composer dan pemuatan aset CDN.

B.2 Perangkat Lunak

- macOS.
- XAMPP: Apache, MariaDB 10.4.28, dan phpMyAdmin.
- PHP 8.5.0.
- Laravel Framework 13.38.0.
- Composer 2.8.12.
- Node.js v24.15.0 dan NPM 11.12.1 (tersedia, tetapi tidak digunakan karena aset memakai CDN).
- Visual Studio Code.
- Google Chrome atau Safari.
- Git.
- Bootstrap 5.3.0 dan Font Awesome 6.4.0 melalui CDN.

Project ini **tidak menggunakan Laravel Breeze**. Authentication dibuat manual di dalam folder `app/Http/Controllers/Auth`. Dengan demikian, tidak ada `routes/auth.php`, tidak ada dependensi Breeze, dan tidak diperlukan build Vite untuk halaman authentication maupun LaporBanjir. Ketiadaan Breeze dan Vite juga membuat Node.js/NPM tidak wajib dijalankan pada project ini.

B.3 Konfigurasi Project

- Lokasi project: `TA_Modul6`.
- Database: `db_laporanbanjir`.
- Host: `127.0.0.1`.
- Port: `3307`.

Isi file `.env` yang digunakan:

```
APP_NAME=Laravel
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3307
DB_DATABASE=db_laporanbanjir
DB_USERNAME=root
DB_PASSWORD=

SESSION_DRIVER=database
```

Konfigurasi tersebut menghasilkan empat tabel utama:

Tabel	Fungsi
`users`	Menyimpan akun petugas.
`password_reset_tokens`	Menyimpan token reset password bawaan Laravel.
`sessions`	Menyimpan session login karena `SESSION_DRIVER=database`.
`laporan`	Menyimpan laporan banjir dari warga.

C. Langkah-Langkah Praktikum

C.1 Persiapan Project dan Database

- Jalankan `composer install`.
- Siapkan file `.env` dan sesuaikan koneksi database.
- Jalankan `php artisan key:generate`.
- Jalankan migration untuk membuat tabel.
- Jalankan seeder untuk mengisi akun petugas dan data laporan.

```
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate
php artisan db:seed
```

Untuk penyiapan ulang dari kondisi bersih, digunakan:

```
php artisan migrate:fresh --seed
```

Perintah tersebut menghasilkan:

- 5 akun petugas dummy.
- 1 akun admin.
- 15 data laporan banjir.

Tabel `sessions` wajib ada karena project memakai `SESSION_DRIVER=database`. Tanpa tabel tersebut, status login tidak dapat dipertahankan.

C.2 Pembuatan Authentication Manual

Project ini memakai jalur **authentication manual tanpa Laravel Breeze**. Pilihan ini diambil karena tugas LaporBanjir pada Modul 6 meminta fitur register, login, dan logout dibuat tanpa Breeze, sehingga alur validasi, hashing, session, dan middleware dapat terlihat langsung.

File utama yang digunakan:

- `app/Http/Controllers/Auth/RegisterController.php`
- `app/Http/Controllers/Auth/LoginController.php`
- `app/Http/Controllers/HomeController.php`
- `routes/web.php`
- `resources/views/auth/login.blade.php`
- `resources/views/auth/register.blade.php`
- `resources/views/layouts/app.blade.php`

Modul 6 memang juga membahas Laravel Breeze pada subbab 6.4.5, tetapi subbab 6.4.10 menyediakan jalur manual. Untuk tugas LaporBanjir, jalur manual yang dipakai agar tidak terjadi tumpang tindih route dan view antara Breeze dan authentication buatan sendiri.

C.3 Membuat RegisterController

- File: `app/Http/Controllers/Auth/RegisterController.php`.
- `create()` menampilkan form registrasi.
- `store()` memvalidasi input, membuat user, lalu langsung login.

```
public function store(Request $request)
{
    $request->validate([
        'name' => ['required', 'string', 'max:255'],
        'email' => ['required', 'string', 'email', 'max:255', 'unique:users'],
        'password' => ['required', 'confirmed', Rules\Password::defaults()],
    ]);

    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
    ]);

    event(new Registered($user));

    Auth::login($user);

    return redirect()->route('dashboard');
}
```

- `unique:users` mencegah email ganda.
- `confirmed` mewajibkan input `password_confirmation`.
- `Hash::make()` mengubah password menjadi hash bcrypt.
- `Auth::login()` langsung membuat session login.
- Pengguna diarahkan ke dashboard setelah registrasi berhasil.

Pada tampilan, halaman ini diberi label **Registrasi Petugas** dan catatan bahwa warga tidak perlu mendaftar.

C.4 Membuat LoginController

- File: `app/Http/Controllers/Auth/LoginController.php`.
- `create()` menampilkan form login.
- `store()` memverifikasi email dan password.
- `destroy()` menghapus session login.

```
public function store(Request $request)
{
    $credentials = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required'],
    ]);

    if (Auth::attempt($credentials, $request->remember)) {
        $request->session()->regenerate();

        return redirect()->intended('dashboard');
    }

    return back()->withErrors([
        'email' => 'Email atau password yang Anda masukkan salah.',
    ]);
}

public function destroy(Request $request)
{
    Auth::logout();

    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return redirect('/');
}
```

- `Auth::attempt()` mencocokkan email dan password dengan hash di database.
- Parameter `remember` mengaktifkan fitur Remember Me.
- `session()->regenerate()` mencegah session fixation.
- `intended()` mengarahkan pengguna ke halaman tujuan atau dashboard.
- Logout menghapus session dan mengganti token CSRF.

Halaman login diberi label **Login Petugas** agar alurnya jelas berbeda dari form warga.

C.5 Pengaturan Route dan Middleware

File: `routes/web.php`.

```
Route::get('/', function () {
    return view('welcome');
});

Route::get('/laporbanjir', [LaporanController::class, 'form'])->name('laporbanjir.form');
Route::post('/laporbanjir/simpan', [LaporanController::class, 'simpan'])-
>name('laporbanjir.simpan');

Route::get('/register', [RegisterController::class, 'create'])->name('register');
Route::post('/register', [RegisterController::class, 'store']);

Route::get('/login', [LoginController::class, 'create'])->name('login');
Route::post('/login', [LoginController::class, 'store']);

Route::post('/logout', [LoginController::class, 'destroy'])->name('logout');

Route::middleware('auth')->group(function () {
    Route::get('/dashboard', [HomeController::class, 'dashboard'])->name('dashboard');
    Route::get('/profile', [HomeController::class, 'profile'])->name('profile');
    Route::get('/daftar-laporan', [LaporanController::class, 'daftar'])-
>name('laporan.daftar');
});
```

Ringkasan route:

Method	URI	Name	Controller	Akses
GET	`/`	-	Closure	Publik
GET	`/laporbanjir`	`laporbanjir.form`	`LaporanController@form`	Publik
POST	`/laporbanjir/simpan`	`laporbanjir.simpan`	`LaporanController@simpan`	Publik
GET	`/register`	`register`	`RegisterController@create`	Publik
POST	`/register`	-	`RegisterController@store`	Publik
GET	`/login`	`login`	`LoginController@create`	Publik
POST	`/login`	-	`LoginController@store`	Publik
POST	`/logout`	`logout`	`LoginController@destroy`	Publik
GET	`/dashboard`	`dashboard`	`HomeController@dashboard`	`auth`
GET	`/profile`	`profile`	`HomeController@profile`	`auth`
GET	`/daftar-laporan`	`laporan.daftar`	`LaporanController@daftar`	`auth`

Halaman **dashboard**, **profil**, dan **daftar laporan** dilindungi middleware `auth`. Jika dibuka tanpa login, pengguna diarahkan ke route `login`. Route `login` wajib memiliki nama `login` agar redirect middleware bekerja.

Pada implementasi ini, route login dan register tidak dibungkus middleware `guest`. Pengguna yang sudah login masih dapat membuka halaman login/register secara langsung. Pembatasan utama tetap berada pada middleware `auth` untuk halaman petugas.

C.6 Membuat Layout dan Navigasi

- File: `resources/views/layouts/app.blade.php`.
- Layout memakai Bootstrap 5.3.0 dan Font Awesome 6.4.0 melalui CDN.
- Menu berubah otomatis dengan directive `@auth` dan `@else`.
- Logout dibuat dalam form `POST` dengan `@csrf`.

```
@auth
    <a href="{{ route('dashboard') }}">Dashboard</a>
    <a href="{{ route('laporan.daftar') }}">Daftar Laporan</a>
    <a href="{{ route('profile') }}">Profil Petugas</a>
    <form method="POST" action="{{ route('logout') }}">
        @csrf
        <button type="submit">Logout</button>
    </form>
@else
    <a href="{{ route('laporbanjir.form') }}">Lapor Banjir</a>
    <a href="{{ route('login') }}">Login Petugas</a>
    <a href="{{ route('register') }}">Registrasi Petugas</a>
@endauth
```

- Guest melihat menu **Lapor Banjir**, **Login Petugas**, dan **Registrasi Petugas**.
- Petugas yang sudah login melihat **Dashboard**, **Daftar Laporan**, **Profil Petugas**, dan **Logout**.
- Layout juga menampilkan pesan sukses, pesan error, dan error validasi.
- `@yield('content')` menjadi tempat isi halaman.

Halaman `welcome` juga ditambahkan tautan **Lapor Banjir**, **Login Petugas**, dan **Registrasi Petugas** pada bagian navigasi. Halaman awal masih menggunakan tampilan bawaan Laravel, tetapi pintu masuk warga dan petugas sudah dipisahkan.

C.7 Membuat Halaman Dashboard dan Profil

- File controller: `app/Http/Controllers/HomeController.php`.
- `dashboard()` menampilkan `resources/views/dashboard.blade.php`.
- `profile()` menampilkan `resources/views/profile.blade.php`.

```
public function dashboard()
{
    return view('dashboard');
}

public function profile()
{
    return view('profile');
}
```

Dashboard menampilkan pesan konfirmasi bahwa pengguna sudah login. Halaman profil menampilkan data pengguna yang sedang login:

```
<p>Name: {{ Auth::user()->name }}</p>
<p>Email: {{ Auth::user()->email }}</p>
<p>Account created: {{ Auth::user()->created_at->diffForHumans() }}</p>
```

- `Auth::user()` mengambil data pengguna dari session.
- `diffForHumans()` menampilkan waktu pembuatan akun dalam format yang mudah dibaca.
- Kedua halaman berada di dalam middleware `auth`.

C.8 Membuat Seeder Akun Petugas

- File: `database/seeders/PetugasSeeder.php`.
- Faker menghasilkan nama dan email acak.
- `unique()` mencegah email ganda.
- Password di-hash dengan `Hash::make()`.
- Seeder membuat 5 akun dummy dan 1 akun admin.

```
$faker = Faker::create('id_ID');

for ($i = 0; $i < 5; $i++) {
    DB::table('users')->insert([
        'name' => $faker->name,
        'email' => $faker->unique()->safeEmail,
        'password' => Hash::make('password'),
        'created_at' => now(),
        'updated_at' => now(),
    ]);
}

DB::table('users')->insert([
    'name' => 'Admin BPBD',
    'email' => 'admin@example.com',
    'password' => Hash::make('admin123'),
    'created_at' => now(),
    'updated_at' => now(),
]);
```

Akun yang dihasilkan:

Akun	Email	Password	Keterangan
Admin BPBD	`admin@example.com`	`admin123`	Akun admin untuk pengujian.
5 akun dummy	Email acak dari Faker	`password`	Akun petugas dummy.

`DatabaseSeeder` memanggil dua seeder:

```
$this->call([
    PetugasSeeder::class,
    LaporanSeeder::class,
]);
```

Dengan demikian, satu perintah `php artisan db:seed` mengisi akun petugas sekaligus data laporan.

C.9 Integrasi Fitur LaporBanjir dari Modul 4

Fitur LaporBanjir dipindahkan dari project Modul 4 ke project Modul 6. Bagian yang diintegrasikan meliputi migration, model, controller, view, dan seeder.

C.9.1 Migration Tabel Laporan

File: `database/migrations/2026_10_06_044521_create_laporan_table.php`.

```
Schema::create('laporan', function (Blueprint $table) {
    $table->id();
    $table->string('nama_pelapor');
    $table->string('lokasi');
    $table->string('tinggi_genangan');
    $table->string('tanggal_kejadian');
    $table->timestamps();
});
```

Kolom `tinggi_genangan` dan `tanggal_kejadian` disimpan sebagai string agar sesuai dengan implementasi Modul 4.

C.9.2 Model Laporan

File: `app/Models/Laporan.php`.

```
class Laporan extends Model
{
    protected $table = 'laporan';

    protected $fillable = [
        'nama_pelapor',
        'lokasi',
        'tinggi_genangan',
        'tanggal_kejadian',
    ];
}
```

- `$table` menegaskan nama tabel tunggal `laporan`.
- `$fillable` mengizinkan mass assignment pada kolom yang digunakan form.

C.9.3 LaporanController

File: `app/Http/Controllers/LaporanController.php`.

```
public function form()
{
    return view('form-laporan');
}

public function simpan(Request $request)
{
    $request->validate([
        'nama_pelapor' => 'required',
        'lokasi' => 'required',
        'tinggi_genangan' => 'required',
        'tanggal_kejadian' => 'required|date',
    ]);

    Laporan::create([
        'nama_pelapor' => $request->nama_pelapor,
        'lokasi' => $request->lokasi,
        'tinggi_genangan' => $request->tinggi_genangan,
        'tanggal_kejadian' => $request->tanggal_kejadian,
    ]);

    return view('hasil-laporan');
}

public function daftar()
{
    $laporan = Laporan::all();

    return view('daftar-laporan', compact('laporan'));
}
```

- `form()` menampilkan form laporan untuk warga.
- `simpan()` memvalidasi input lalu menyimpan laporan ke database.
- `daftar()` mengambil semua laporan untuk halaman petugas.
- Form laporan bersifat publik dan tidak memerlukan login.

C.9.4 Seeder Laporan

File: `database/seeders/LaporanSeeder.php`.

```
$lokasi = [
    'Dayeuhkolot',
    'Baleendah',
    'Bojongsoang',
    'Rancaekek',
    'Majalaya',
];

for ($i = 0; $i < 15; $i++) {
    DB::table('laporan')->insert([
        'nama_pelapor' => $faker->name,
        'lokasi' => $faker->randomElement($lokasi),
        'tinggi_genangan' => $faker->numberBetween(10, 150),
        'tanggal_kejadian' => $faker->dateTimeBetween('-1 month', 'now')->format('Y-m-d'),
        'created_at' => now(),
        'updated_at' => now(),
    ]);
}
```

Seeder menghasilkan 15 laporan dengan lokasi di kawasan Bandung dan sekitarnya.

C.9.5 Status Laporan

Halaman `daftar-laporan.blade.php` menentukan status berdasarkan tinggi genangan:

Tinggi Genangan	Status
`<= 50 cm`	Waspada
`51-100 cm`	Siaga
`> 100 cm`	Awas

Logika tersebut membuat daftar laporan lebih mudah dibaca oleh petugas.

C.10 Pembedaan Alur Warga dan Petugas

Pembedaan dilakukan pada tingkat tampilan dan akses halaman:

Aspek	Warga	Petugas
Akun	Tidak memiliki akun.	Memiliki akun di tabel `users`.
Registrasi	Tidak perlu registrasi.	Melalui **Registrasi Petugas**.
Login	Tidak perlu login.	Melalui **Login Petugas**.
Pintu masuk	Menu **Lapor Banjir**.	Menu **Login Petugas**.
Halaman utama	Form `/laporbanjir`.	Dashboard, daftar laporan, profil.
Melihat daftar laporan	Tidak dapat mengakses `/daftar-laporan`.	Dapat mengakses `/daftar-laporan`.
Catatan pada form	“Anda melapor sebagai warga tanpa login.”	“Anda sedang login sebagai petugas; form ini adalah mode warga.”

Dengan pembedaan ini:

- Warga tetap dapat melapor tanpa membuat akun.
- Petugas memiliki area khusus untuk melihat seluruh laporan.
- Petugas yang membuka form warga tetap melihat catatan bahwa form tersebut adalah mode warga.
- Menu petugas tidak menampilkan tautan **Lapor Banjir**, sedangkan menu guest menampilkannya.

Perlu dicatat bahwa petugas masih dapat membuka `/laporbanjir` secara langsung. Form tersebut tetap berfungsi dan laporan yang dikirim tidak dikaitkan dengan akun petugas.

C.11 Pengujian

Jalankan server:

```
php artisan serve
```

Skenario pengujian yang dilakukan:

- Buka `/` sebagai guest; pastikan menu guest tampil.
- Buka `/dashboard` tanpa login; pastikan diarahkan ke `/login`.
- Registrasi akun baru; pastikan akun dibuat dan langsung masuk dashboard.
- Login dengan password salah; pastikan pesan galat tampil.
- Login `admin@example.com` / `admin123`; pastikan masuk dashboard.
- Buka `/profile`; pastikan nama, email, dan waktu pembuatan akun tampil.
- Logout; pastikan session dihapus dan kembali ke halaman awal.
- Kirim laporan sebagai guest; pastikan laporan tersimpan dan halaman hasil tampil.
- Buka `/daftar-laporan` sebagai guest; pastikan diarahkan ke `/login`.
- Login sebagai petugas dan buka `/daftar-laporan`; pastikan 15 laporan tampil.
- Buka `/laporbanjir` sebagai petugas; pastikan catatan mode warga tampil.

Hasil pengujian otomatis dan pemeriksaan kode:

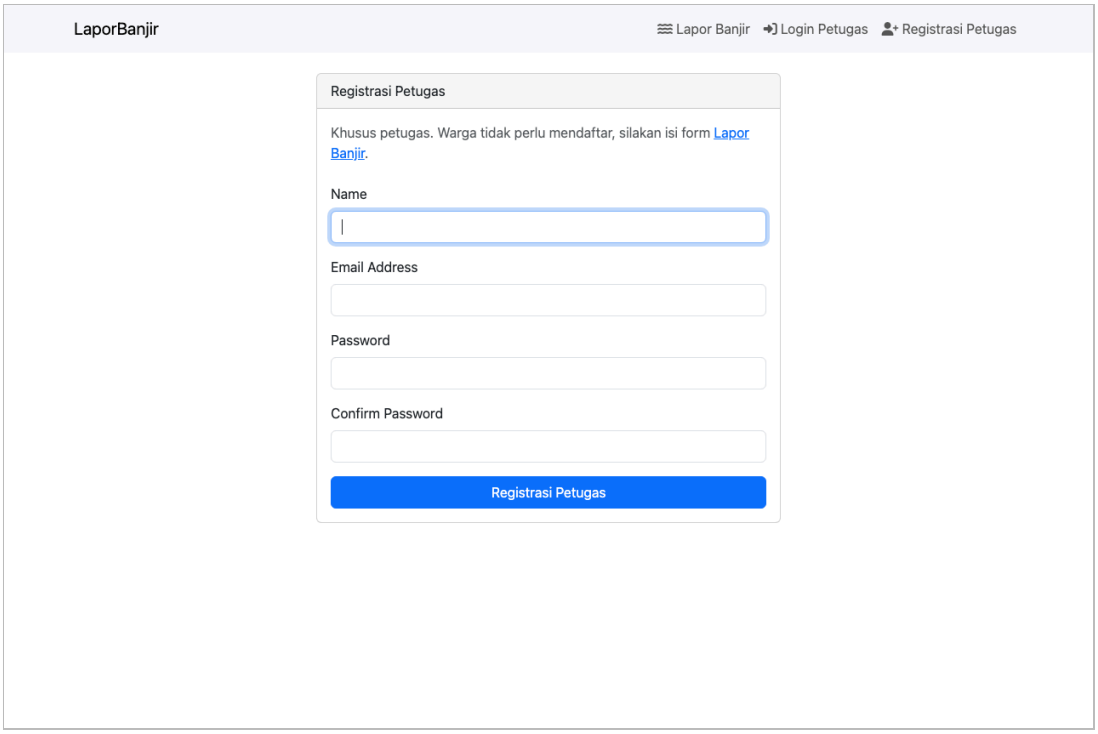
- `php artisan test`: **2 test lulus, 2 assertion**.
- `./vendor/bin/pint --test`: **lulus**.
- `php artisan view:cache` dan `php artisan view:clear`: **berhasil tanpa error**.

Data uji `juu@gmail.com` dan laporan “Uji Screenshot” dihapus kembali. Data akhir project: **6 user** (5 dummy + 1 admin) dan **15 laporan**. Verifikasi tabel `users` dan `laporan` dilakukan melalui phpMyAdmin; kolom `password` berisi hash, bukan password asli.

D. Hasil dan Dokumentasi Program

Bagian ini menampilkan hasil pengujian fitur authentication dan integrasi LaporBanjir. Pengujian dilakukan melalui browser dengan data akun hasil seeder.

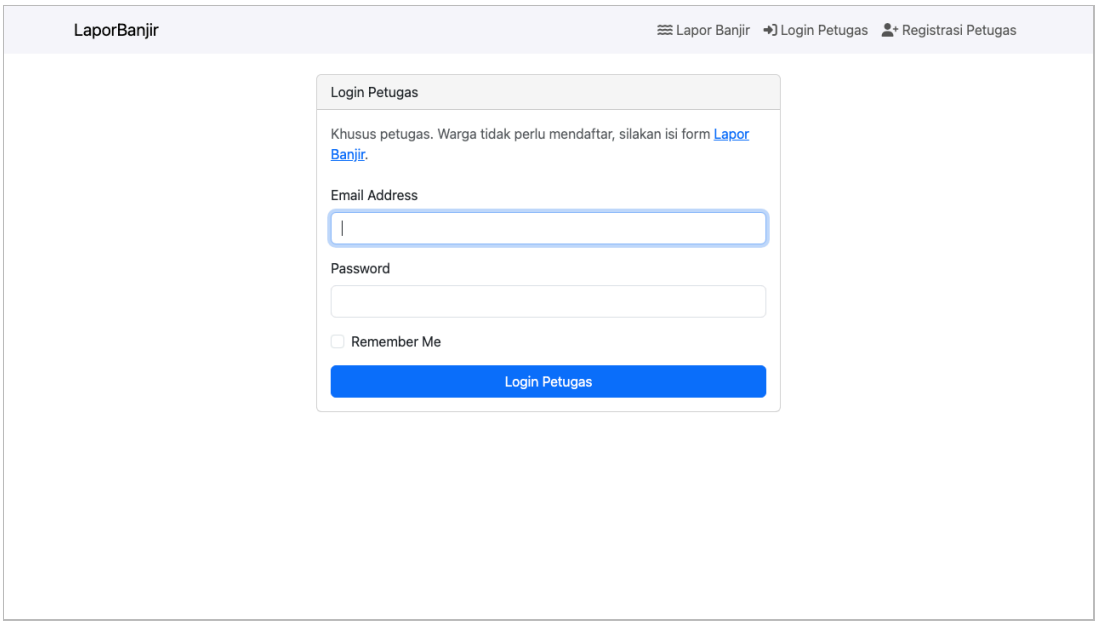
D.1 Halaman Registrasi Petugas



Gambar 1. *Halaman registrasi untuk membuat akun petugas.*

- Form berisi nama, email, password, dan konfirmasi password.
- Terdapat catatan bahwa warga tidak perlu mendaftar.
- Tautan menuju form Lapor Banjir tersedia untuk warga.

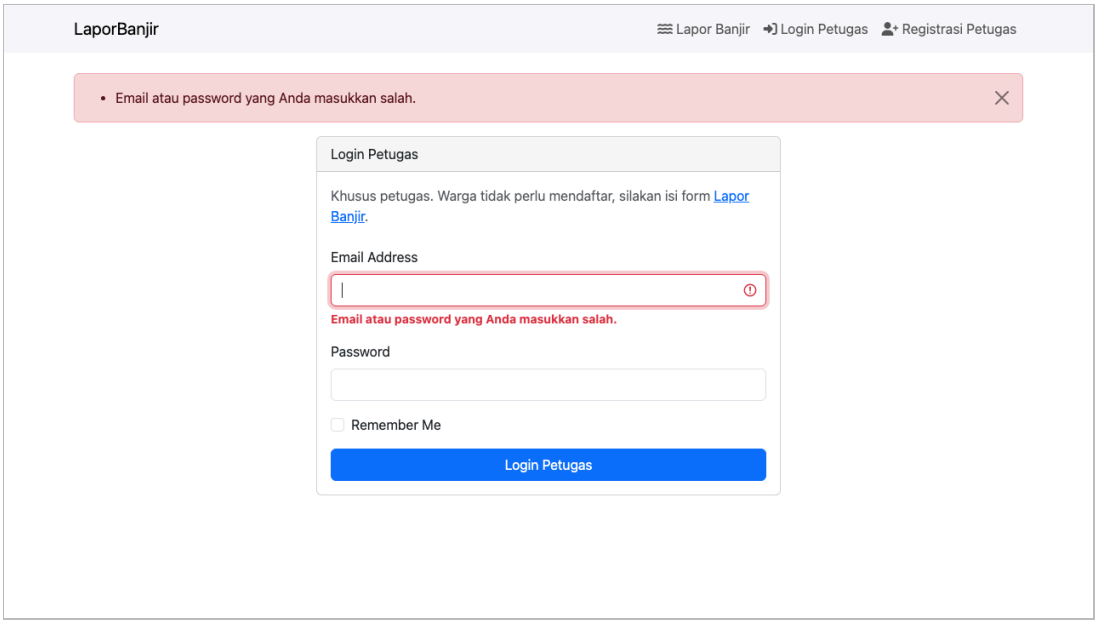
D.2 Halaman Login Petugas



Gambar 2. *Halaman login petugas.*

- Form meminta email dan password.
- Tersedia checkbox Remember Me.
- Halaman diberi label **Login Petugas** agar tidak tertukar dengan alur warga.

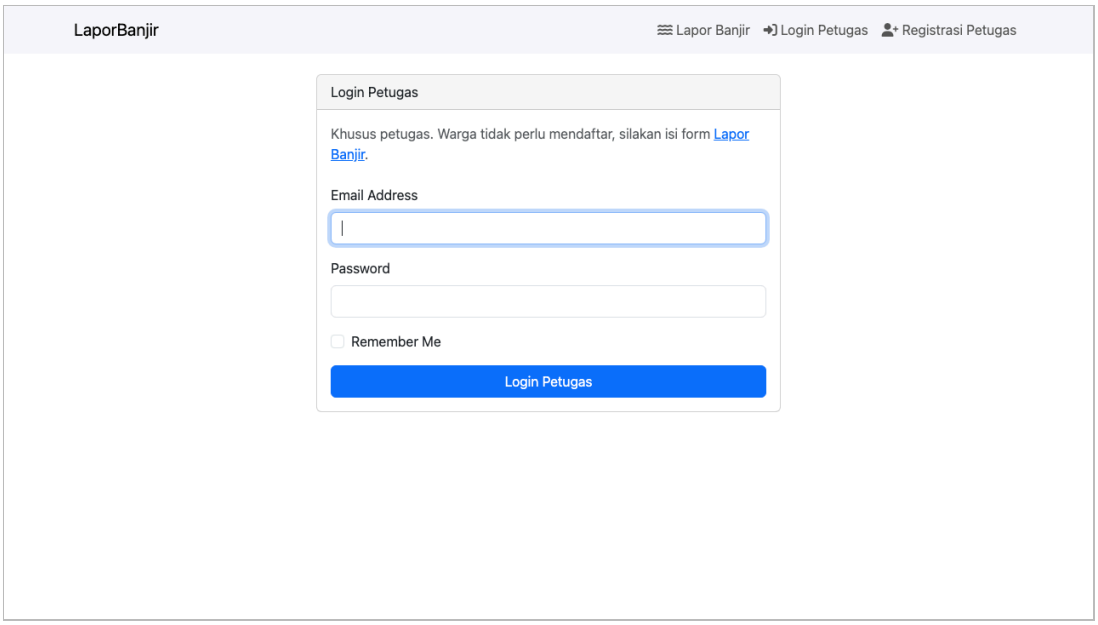
D.3 Pesan Galat Login



Gambar 3. *Pesan galat ketika kredensial salah.*

- Pesan muncul pada alert dan di bawah input email.
- Pengguna tetap berada di halaman login.

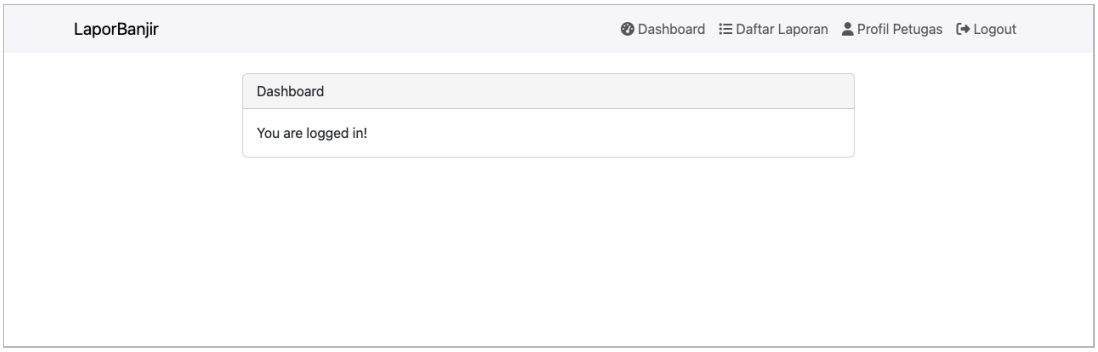
D.4 Redirect ke Halaman Login



Gambar 4. *Redirect saat membuka dashboard tanpa login.*

- Middleware `auth` mencegah akses.
- Pengguna diarahkan ke route `login`.
- Setelah login, pengguna dapat diarahkan kembali ke halaman yang dituju.

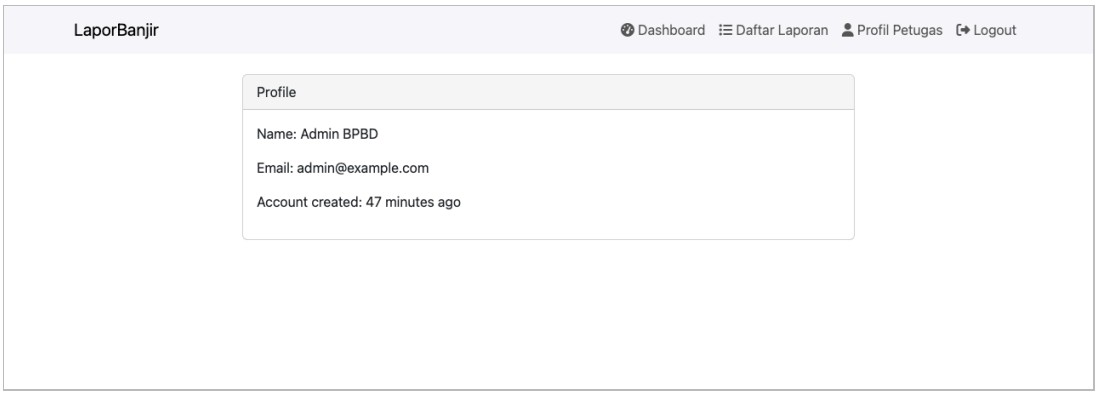
D.5 Dashboard Setelah Login



Gambar 5. *Dashboard setelah petugas berhasil login.*

- Dashboard menampilkan konfirmasi bahwa pengguna sudah login.
- Menu berubah menjadi Dashboard, Daftar Laporan, Profil Petugas, dan Logout.
- Halaman dilindungi middleware `auth`.
- Setelah logout, session dihapus dan tampilan kembali ke mode guest.

D.6 Profil Petugas



Gambar 6. *Halaman profil petugas.*

- Profil menampilkan nama, email, dan waktu pembuatan akun.
- Data diambil dari `Auth::user()`.
- Waktu akun dibuat ditampilkan dengan `diffForHumans()`.

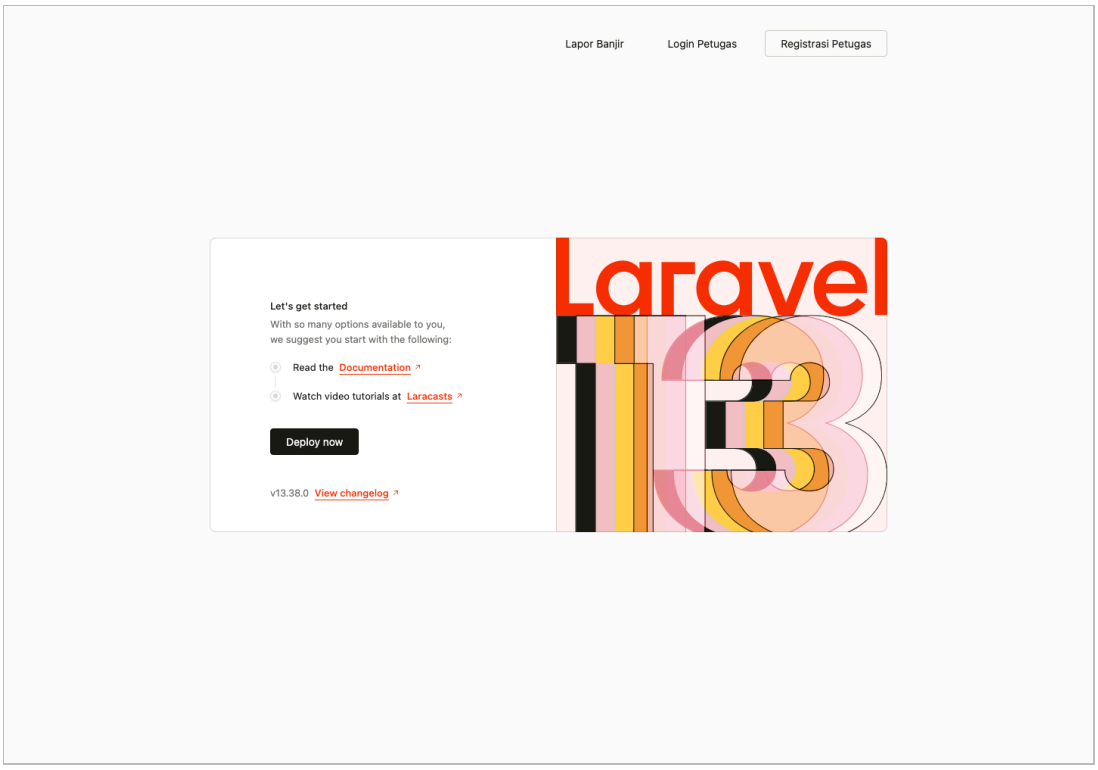
D.7 Tabel Users Hasil Seeder

	id	name	email	email_verified_at	password	remember_token	created_at	updated_at
Edit Copy Delete	1	Koko Aswani Hutagalung	maria50@example.net	NULL	$2y$12$.wMb74TQ0p30nsFg7LfmueSxvDai/hSkjRkhEgIAB0c...	NULL	2026-10-21 17:55:19	2026-10-21 17:55:19
Edit Copy Delete	2	Asman Asirwada Latupono	hpalastri@example.com	NULL	$2y$12$kDcEs1J7uH8leg5iZS9AYOTSMMFfV158fyqANz82FYu...	NULL	2026-10-21 17:55:19	2026-10-21 17:55:19
Edit Copy Delete	3	Yani Puspita	ana19@example.net	NULL	$2y$12$UZp4qHNWFWWHeprjvh/qsecC3/WAeSUy/NYMoUEOvAI...	NULL	2026-10-21 17:55:19	2026-10-21 17:55:19
Edit Copy Delete	4	Balijan Asirwada Firmansyah S.Gz	ibun.wahyuni@example.net	NULL	$2y$12$B5RdQFJ3MMykzaUUCOgFLONtqK7RzGyws0ncbcvHjya...	NULL	2026-10-21 17:55:19	2026-10-21 17:55:19
Edit Copy Delete	5	Yusuf Hutasoit	vsitumorang@example.com	NULL	$2y$12$7ZQJ3/rm3SVkKj/HY2Ss8ugxNLd/z1qaVMqwqQ4bwTM...	NULL	2026-10-21 17:55:20	2026-10-21 17:55:20
Edit Copy Delete	6	Admin BPBD	admin@example.com	NULL	$2y$12$YHCx4INihVDVLlq60HZR9Og65L1LGhThZDgrr0TMVrp...	NULL	2026-10-21 17:55:20	2026-10-21 17:55:20

Gambar 7. *Data akun hasil seeder pada tabel `users` melalui phpMyAdmin.*

- Tabel berisi 6 baris: 5 akun dummy dengan nama Faker dan 1 akun `Admin BPBD` (`admin@example.com`).
- Kolom yang tampak: `id`, `name`, `email`, `email_verified_at`, `password`, `remember_token`, `created_at`, dan `updated_at`.
- Kolom `password` berisi hash bcrypt `$2y$12$...`; password asli tidak ditampilkan.
- Data dibuat pada 2026-10-21 pukul 17:55:19-17:55:20.

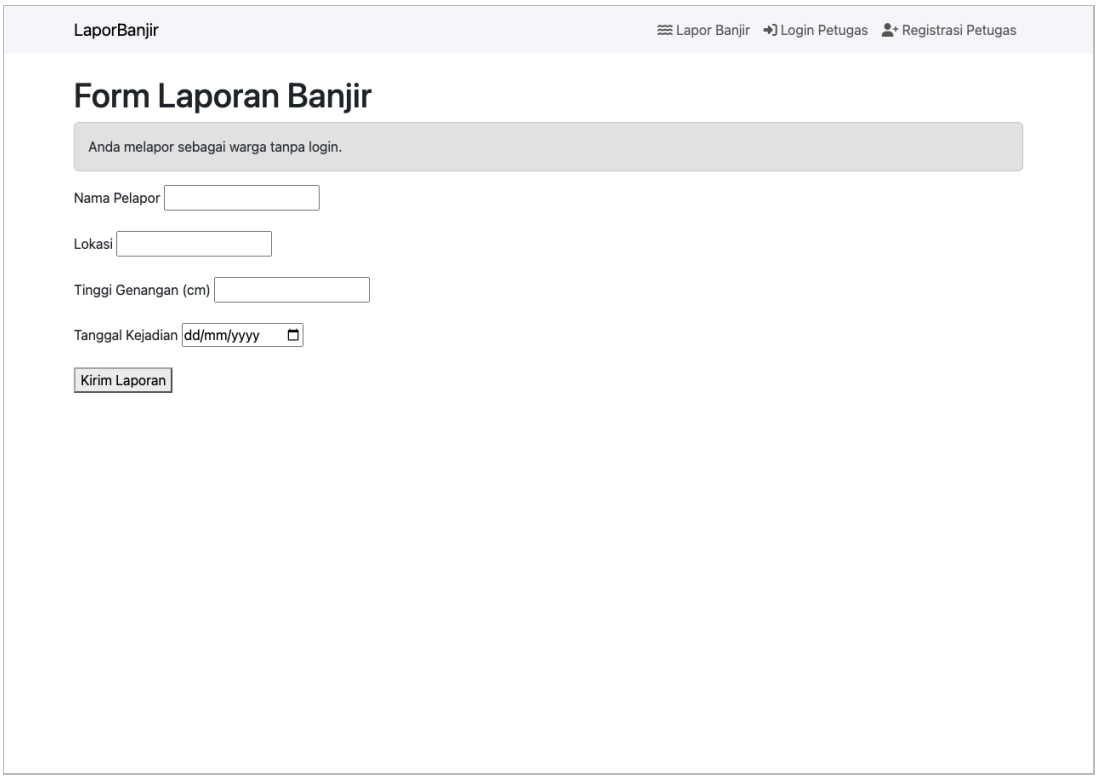
D.8 Halaman Awal



Gambar 8. *Halaman awal dengan navigasi warga dan petugas.*

- Guest melihat **Lapor Banjir**, **Login Petugas**, dan **Registrasi Petugas**.
- Halaman awal menggunakan tampilan bawaan Laravel dengan tambahan navigasi project.
- Warna halaman mengikuti preferensi tema sistem; dokumentasi ini memakai mode terang agar konsisten.

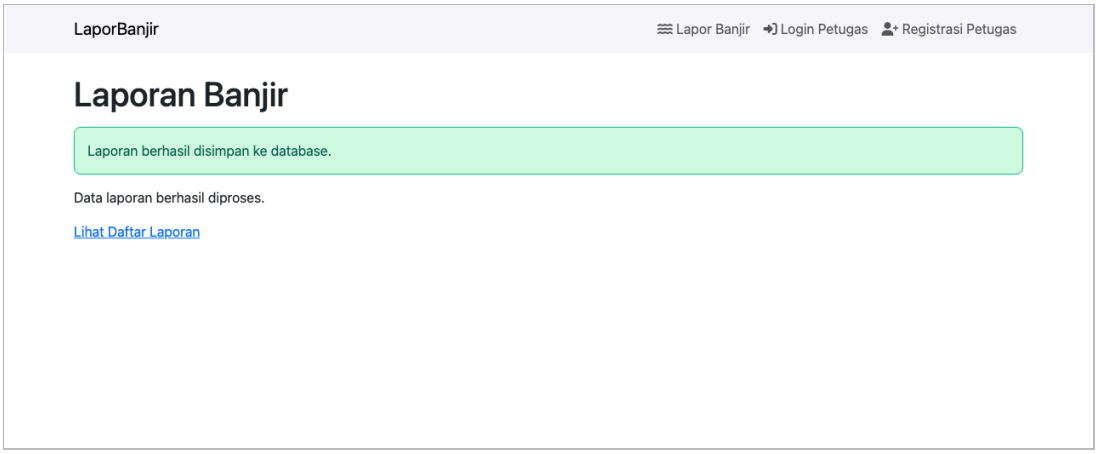
D.9 Form Laporan Warga



Gambar 9. *Form laporan untuk warga tanpa login.*

- Form berisi nama pelapor, lokasi, tinggi genangan, dan tanggal kejadian.
- Muncul catatan “Anda melapor sebagai warga tanpa login.”
- Form dapat diakses tanpa akun.

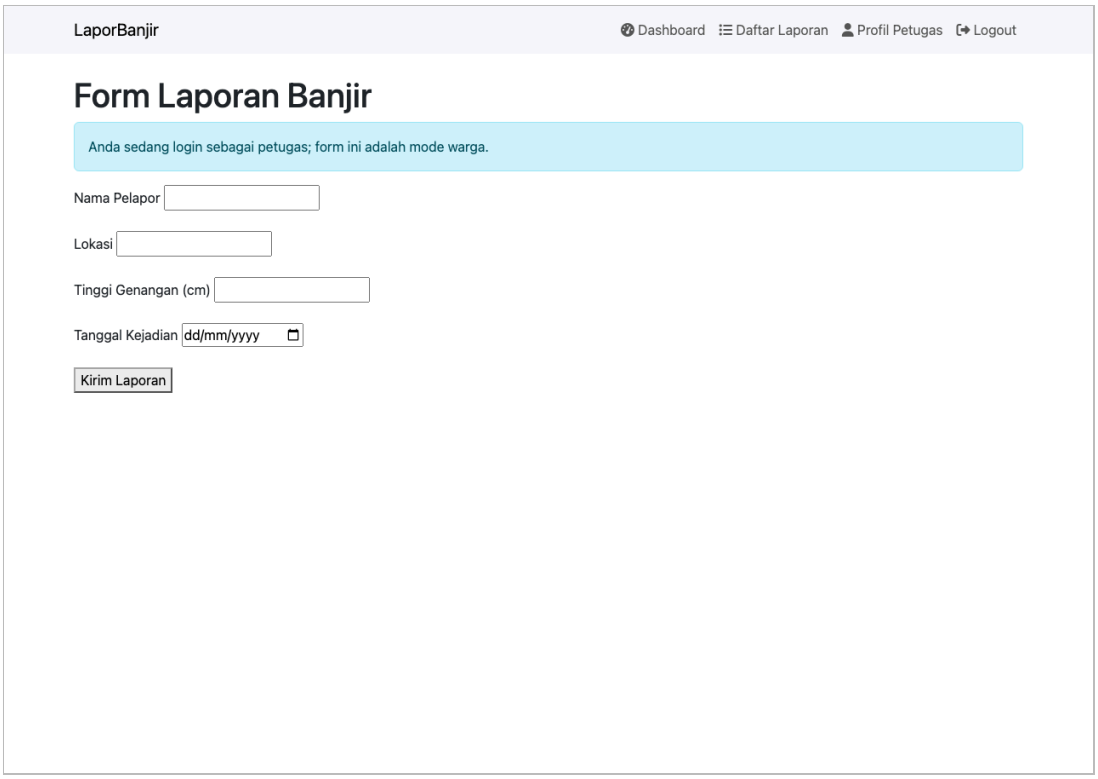
D.10 Laporan Berhasil Disimpan



Gambar 10. *Halaman hasil setelah laporan dikirim.*

- Laporan berhasil masuk ke tabel `laporan`.
- Pesan sukses ditampilkan melalui komponen `x-alert`.

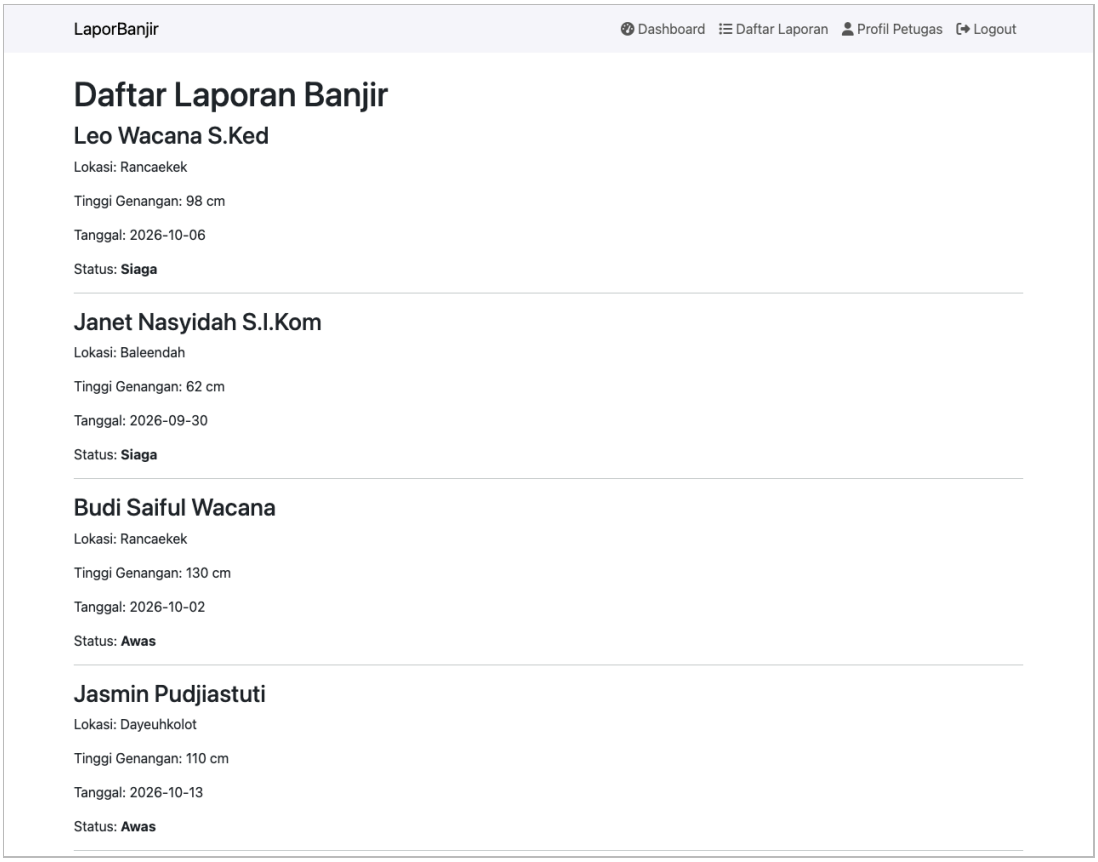
D.11 Form Laporan Saat Login sebagai Petugas



Gambar 11. *Form laporan saat dibuka oleh petugas.*

- Form tetap dapat dibuka oleh petugas.
- Muncul catatan “Anda sedang login sebagai petugas; form ini adalah mode warga.”
- Laporan yang dikirim melalui form ini tidak dikaitkan dengan akun petugas.

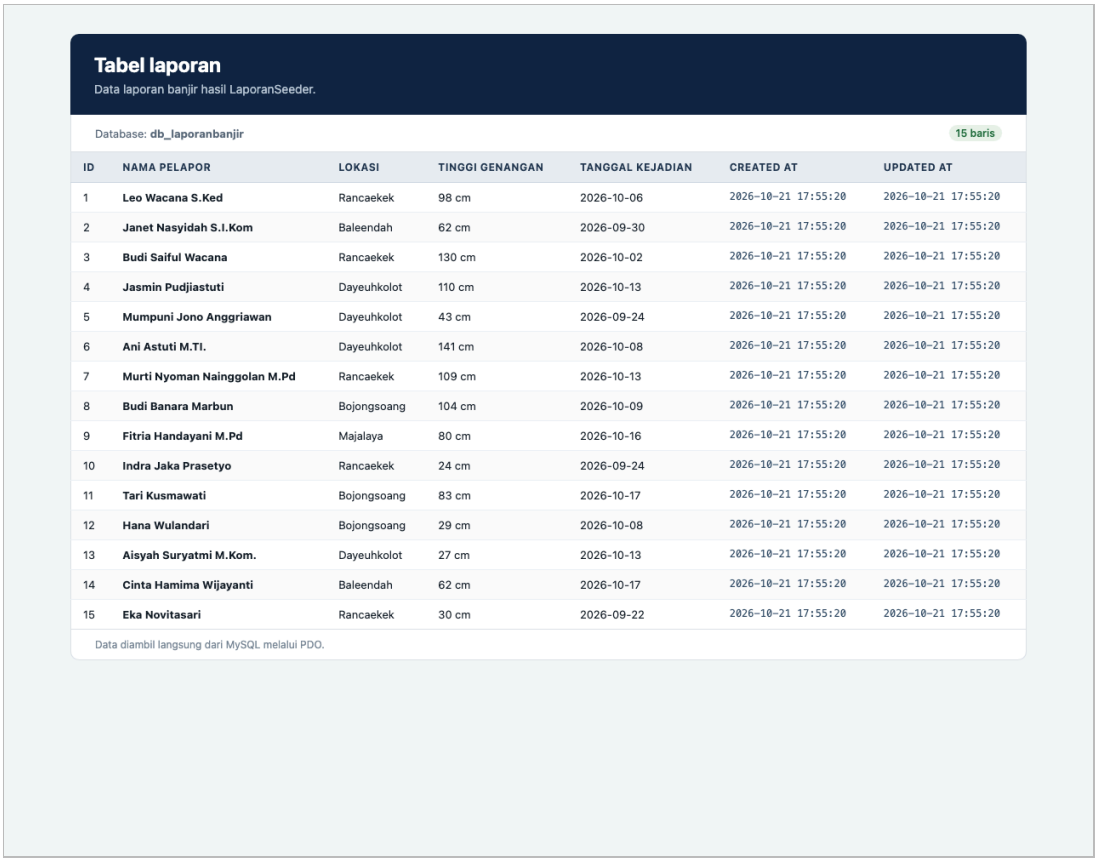
D.12 Daftar Laporan Petugas



Gambar 12. *Bagian awal halaman daftar laporan petugas.*

- Halaman menampilkan nama pelapor, lokasi, tinggi genangan, tanggal, dan status.
- Status ditentukan dari tinggi genangan: Waspada, Siaga, atau Awas.
- Halaman hanya dapat diakses setelah login.

D.13 Tabel Laporan



Tabel laporan

Data laporan banjir hasil LaporanSeeder.

Database: **db_laporanbanjir** — 15 baris

ID	NAMA PELAPOR	LOKASI	TINGGI GENANGAN	TANGGAL KEJADIAN	CREATED AT	UPDATED AT
1	**Leo Wacana S.Ked**	Rancaekek	98 cm	2026-10-06	2026-10-21 17:55:20	2026-10-21 17:55:20
2	**Janet Nasyidah S.I.Kom**	Baleendah	62 cm	2026-09-30	2026-10-21 17:55:20	2026-10-21 17:55:20
3	**Budi Saiful Wacana**	Rancaekek	130 cm	2026-10-02	2026-10-21 17:55:20	2026-10-21 17:55:20
4	**Jasmin Pudjiastuti**	Dayeuhkolot	110 cm	2026-10-13	2026-10-21 17:55:20	2026-10-21 17:55:20
5	**Mumpuni Jono Anggriawan**	Dayeuhkolot	43 cm	2026-09-24	2026-10-21 17:55:20	2026-10-21 17:55:20
6	**Ani Astuti M.TI.**	Dayeuhkolot	141 cm	2026-10-08	2026-10-21 17:55:20	2026-10-21 17:55:20
7	**Murti Nyoman Nainggolan M.Pd**	Rancaekek	109 cm	2026-10-13	2026-10-21 17:55:20	2026-10-21 17:55:20
8	**Budi Banara Marbun**	Bojongsoang	104 cm	2026-10-09	2026-10-21 17:55:20	2026-10-21 17:55:20
9	**Fitria Handayani M.Pd**	Majalaya	80 cm	2026-10-16	2026-10-21 17:55:20	2026-10-21 17:55:20
10	**Indra Jaka Prasetyo**	Rancaekek	24 cm	2026-09-24	2026-10-21 17:55:20	2026-10-21 17:55:20
11	**Tari Kusmawati**	Bojongsoang	83 cm	2026-10-17	2026-10-21 17:55:20	2026-10-21 17:55:20
12	**Hana Wulandari**	Bojongsoang	29 cm	2026-10-08	2026-10-21 17:55:20	2026-10-21 17:55:20
13	**Aisyah Suryatmi M.Kom.**	Dayeuhkolot	27 cm	2026-10-13	2026-10-21 17:55:20	2026-10-21 17:55:20
14	**Cinta Hamima Wijayanti**	Baleendah	62 cm	2026-10-17	2026-10-21 17:55:20	2026-10-21 17:55:20
15	**Eka Novitasari**	Rancaekek	30 cm	2026-09-22	2026-10-21 17:55:20	2026-10-21 17:55:20

Data diambil langsung dari MySQL melalui PDO.

Gambar 13. *Ringkasan isi tabel* `laporan` *dari database* `db_laporanbanjir`.

- Tabel berisi 15 laporan hasil `LaporanSeeder`.
- Kolom yang tampak: `id`, `nama_pelapor`, `lokasi`, `tinggi_genangan`, `tanggal_kejadian`, `created_at`, dan `updated_at`.
- Lokasi tersebar di Dayeuhkolot, Baleendah, Bojongsoang, Rancaekek, dan Majalaya.
- Tinggi genangan pada data ini berkisar 24-141 cm.
- Tanggal kejadian berkisar 22 September-17 Oktober 2026.

E. Analisis dan Pembahasan

E.1 Perbedaan Aplikasi Sebelum dan Sesudah Authentication

Sebelum authentication diterapkan, seluruh halaman dapat dibuka tanpa identitas pengguna. Setelah authentication diterapkan, aplikasi memiliki pembeda status guest dan petugas yang login.

Aspek	Sebelum Authentication	Sesudah Authentication
Status pengguna	Semua pengunjung dianggap sama.	Guest dan petugas dibedakan.
Akun	Tidak ada akun petugas.	Akun tersimpan di tabel `users`.
Login	Tidak ada proses login.	Login memakai email dan password.
Session	Tidak ada status login.	Status login disimpan di session.
Menu	Menu belum berubah berdasarkan status.	Menu guest dan petugas berbeda.
Dashboard	Dapat dibuka siapa saja.	Hanya dapat dibuka setelah login.
Profil	Tidak ada halaman profil pengguna.	Profil menampilkan data petugas yang login.
Daftar laporan	Dapat diakses tanpa pembatasan.	Dilindungi middleware `auth`.
Logout	Tidak ada proses logout.	Session dihapus dan pengguna kembali menjadi guest.

Perubahan utama terlihat pada navigasi dan akses halaman. Guest tetap dapat mengirim laporan melalui form publik, tetapi halaman pengelolaan laporan hanya tersedia untuk petugas yang sudah login.

E.2 Authentication Manual

Authentication pada project ini dibuat sepenuhnya secara manual:

- Register menggunakan `User::create()` dan `Hash::make()`.
- Login menggunakan `Auth::attempt()`.
- Logout menggunakan `Auth::logout()`, `invalidate()`, dan `regenerateToken()`.
- Halaman petugas dilindungi middleware `auth`.

Pendekatan manual membuat alur authentication terlihat jelas dan mudah dipelajari. Setiap langkah, mulai dari validasi, penyimpanan, pembuatan session, sampai logout, ditulis secara eksplisit.

E.3 Perbandingan Authentication Manual dan Breeze

Aspek	Manual	Breeze
Pembuatan	Ditulis sendiri	Otomatis
Fitur	Sesuai kebutuhan	Lengkap
Pemahaman alur	Lebih jelas	Sebagian disembunyikan
Front-end	Bootstrap CDN	Tailwind dan Vite
Dependensi	Sedikit	Lebih banyak
Risiko konflik	Rendah karena terkontrol	Dapat menimpa route dan view

Modul 6 membahas dua jalur: Breeze pada subbab 6.4.5 dan manual pada subbab 6.4.10. Untuk tugas LaporBanjir, modul secara khusus meminta authentication manual tanpa Breeze. Project ini mengikuti ketentuan tersebut agar alur validasi, hashing, session, dan middleware terlihat langsung. Breeze tidak dipasang sehingga tidak ada route atau view tambahan yang tidak digunakan.

E.4 Peran Session

- Session menyimpan status login pengguna.
- Karena `SESSION_DRIVER=database`, data session disimpan di tabel `sessions`.
- Laravel mengirim cookie session pada setiap request.
- `session()->regenerate()` mencegah session fixation setelah login.
- `invalidate()` menghapus session saat logout.
- `regenerateToken()` mengganti token CSRF setelah logout.

E.5 Hashing Password

- Password tidak disimpan sebagai teks asli.
- `Hash::make()` menghasilkan hash bcrypt.
- Hash tidak dapat dikembalikan menjadi password asli.
- `Auth::attempt()` mencocokkan password dengan hash.
- Password hash diawali `$2y$`.
- Seeder juga memakai `Hash::make()` agar akun dummy dapat login.

E.6 Peran Middleware

- Middleware `auth` melindungi dashboard, profil, dan daftar laporan.
- Pengguna yang belum login diarahkan ke route `login`.
- Route `login` wajib memiliki nama `login` agar redirect bekerja.
- Pada implementasi ini, route login dan register tidak memakai middleware `guest`.
- Penambahan middleware `guest` dapat menjadi pengembangan berikutnya agar pengguna yang sudah login tidak membuka form login lagi.

E.7 Alasan Halaman Wajib Login

Halaman **dashboard**, **profil**, dan **daftar laporan** dipilih sebagai halaman wajib login karena berisi fungsi pengelolaan internal petugas:

- Dashboard menjadi pusat navigasi setelah petugas berhasil login.
- Profil menampilkan data akun yang bersifat pribadi.
- Daftar laporan memuat seluruh laporan warga dan digunakan petugas untuk memantau kondisi banjir.

Sebaliknya, halaman awal dan form **Lapor Banjir** tetap publik karena warga harus dapat melapor tanpa membuat akun. Dengan pembagian ini, akses warga tetap mudah, tetapi data pengelolaan tidak terbuka untuk umum.

E.8 Seeder dan Faker

- Faker menghasilkan nama dan email acak.
- `unique()` mencegah email duplikat.
- `Hash::make()` memastikan akun dummy memiliki password yang valid.
- Seeder mempercepat pembuatan data untuk pengujian.
- `LaporanSeeder` menghasilkan 15 laporan untuk menguji halaman daftar.
- `DatabaseSeeder` memastikan kedua seeder dijalankan dalam satu perintah.

E.9 Integrasi LaporBanjir

- Fitur LaporBanjir dari Modul 4 berhasil diintegrasikan ke Modul 6.
- Form laporan tetap publik.
- Data laporan disimpan ke tabel `laporan`.
- Halaman daftar laporan dilindungi middleware `auth`.
- Status laporan dihitung dari tinggi genangan.
- Integrasi tidak mengubah alur authentication yang sudah dibuat.

E.10 Pembedaan Warga dan Petugas

Pembedaan saat ini berada pada tingkat tampilan dan akses halaman:

- Warga tidak memiliki akun dan melapor melalui form publik.
- Petugas memiliki akun dan dapat melihat seluruh laporan.
- Menu guest dan menu petugas berbeda.
- Form laporan menampilkan catatan sesuai status login.

Namun, pembedaan ini belum sepenuhnya ditegakkan pada tingkat data dan hak akses:

- Tabel `users` belum memiliki kolom `role`.
- Siapa pun yang melakukan registrasi akan menjadi petugas.

- Tabel `laporan` belum memiliki `user_id` atau `jenis_pelapor`.
- Laporan warga dan laporan yang dikirim saat login petugas tidak dapat dibedakan di database.
- Petugas tetap dapat membuka form warga.

Jika ingin pembedaan yang lebih kuat, pengembangan berikutnya dapat menambahkan:

1. Kolom `role` pada tabel `users` (`warga` atau `petugas`).
2. Middleware khusus untuk membatasi area petugas.
3. Kolom `user_id` atau `jenis_pelapor` pada tabel `laporan`.
4. Halaman riwayat laporan untuk warga.
5. Pembatasan registrasi petugas hanya melalui admin.

E.11 Kendala dan Solusi

Kendala	Penyebab	Solusi
Koneksi database gagal	MySQL belum aktif atau port salah	Jalankan MySQL dan gunakan port `3307`.
Status login tidak bertahan	Tabel `sessions` tidak ada	Jalankan migration.
`419 Page Expired`	Token CSRF tidak dikirim atau session berubah	Pastikan `@csrf` ada dan gunakan session yang sama.
`Route [login] not defined`	Route login tidak diberi nama	Tambahkan `->name('login')`.
Login gagal	Password tidak di-hash	Gunakan `Hash::make()`.
Email duplikat	Seeder dijalankan berulang	Gunakan email unik atau reset database.
Data uji tertinggal	Proses pengujian membuat akun/laporan baru	Hapus data uji setelah pengujian.
Tautan daftar laporan tidak dapat dibuka warga	Halaman dilindungi middleware `auth`	Login sebagai petugas terlebih dahulu.

E.12 Catatan Keamanan

- Gunakan `@csrf` pada setiap form.
- Simpan password dalam bentuk hash.
- Gunakan method `POST` untuk logout.
- Regenerasi session setelah login.
- Lindungi halaman petugas dengan middleware.
- Jangan mengunggah file `.env` ke repositori.
- Ganti password admin dengan password yang lebih kuat pada produksi.
- Aktifkan `APP_DEBUG=false` pada produksi.
- Tambahkan rate limiting pada login untuk mencegah brute force.
- Tambahkan validasi role jika aplikasi digunakan oleh lebih dari satu jenis pengguna.

F. Kesimpulan

- Authentication manual berhasil dibuat tanpa Laravel Breeze.
- Register, login, dan logout berjalan sesuai alur yang dirancang.
- `Hash::make()` menjaga password tetap aman.
- `Auth::attempt()` memverifikasi kredensial pengguna.
- Middleware `auth` melindungi dashboard, profil, dan daftar laporan.
- Session login bertahan karena tabel `sessions` tersedia.
- Seeder dan Faker mempercepat pembuatan akun petugas dummy.
- Fitur LaporBanjir dari Modul 4 berhasil diintegrasikan.
- Warga dapat melapor tanpa akun, sedangkan petugas dapat melihat daftar laporan.
- Pembedaan warga dan petugas masih berada pada tingkat tampilan dan akses halaman.
- Pembatasan berbasis role dan pencatatan identitas pelapor dapat ditambahkan pada pengembangan berikutnya.